

BACKTRACKING



UNIVERSIDAD NACIONAL DE INGENIERIA

Facultad de ingeniería Eléctrica y Electrónica

BMA09 Q: Algoritmos y Estructura de Datos I

Ciclo: 2026-2

Docente: Ing. Luis Antonio Selis Vásquez

Presentado por:

Robert Emmanuel Lopez Romero

Freddy Alexander Rodríguez Figueroa

Lima-2026

BACKTRACKING

Marco Teórico.....	3
Concepto de Backtracking	3
Espacio de Búsqueda	3
Construcción de Soluciones.....	4
Mecanismo de Poda (Pruning).....	4
Fuerza Bruta vs. Backtracking.....	5
Algoritmos	5
Programa que genera N permutaciones:	6
Programa para generar y explorar combinaciones de N elementos tomados en grupos de K:	6
Programa que determina la posición de N reinas en un tablero de ajedrez sin que se amenacen entre sí.	7
Programa que resuelve un sudoku:	8
Complejidad computacional de los algoritmos:.....	8
Permutaciones de N elementos	8
Combinaciones (N elementos tomados de K en K)	9
N-Reinas (Tablero de 8 x 8).....	9
Sudoku (9 x 9).....	10
Aplicaciones del Algoritmo de Backtracking	11
Usos de los algoritmos:.....	11
Permutaciones:.....	11
Combinaciones:.....	11
N-Reinas:	11
Sudoku:	11
Comparación entre algoritmos:.....	12
Otras aplicaciones:	13
Problemas de Satisfacción de Restricciones (CSP)	13
Coloreado de Grafos (Graph Coloring)	13
Búsqueda de Rutas y Navegación (Pathfinding y Laberintos)	13
Criptoaritmética y Criptografía.....	13
Pruebas Automáticas y Verificación de Software	13
Bibliografía	14

MARCO TEÓRICO

Concepto de Backtracking

El “Backtracking” es un paradigma algorítmico sistemático enfocado en la resolución de problemas de optimización y satisfacción de restricciones. Su estrategia se basa en una exploración en profundidad (Depth-First Search, DFS) sobre un árbol implícito de posibilidades, donde la solución se construye mediante una secuencia de decisiones incrementales.

Características Principales:

- **Construcción Incremental:** Las soluciones candidatas se construyen paso a paso, añadiendo un elemento al vector de solución parcial en cada nivel de recursión.
- **Evaluación Temprana de Restricciones:** En lugar de esperar a generar una combinación completa para comprobar si es válida, el algoritmo evalúa las restricciones del problema en cada paso intermedio.
- **Mecanismo de Retroceso (Backtrack):** Si una decisión conduce a un estado inválido o a un "callejón sin salida" donde ninguna alternativa futura satisface las condiciones, el algoritmo deshace la última decisión tomada y regresa al estado anterior para probar otras vías inexploradas.
- **Naturaleza Recursiva:** Se implementa comúnmente mediante funciones recursivas que aprovechan la pila de llamadas (call stack) para gestionar el estado del problema de forma implícita.

Espacio de Búsqueda

El espacio de búsqueda es el conjunto de todos los estados o configuraciones posibles que pueden ser explorados durante la resolución de un problema combinatorio.

- **Estructura en Árbol:**
 - Nodo Raíz:** Representa el estado inicial del sistema, la solución vacía o nivel 0.
 - Nodos Intermedios:** Representan soluciones parciales e incompletas en la etapa k.
 - Nodos Hoja:** Representan estados finales del árbol, los cuales pueden ser soluciones válidas completas o estados no válidos descartados.
 - Aristas:** Representan las decisiones válidas o transiciones de estado que llevan de un nodo parcial a su sucesor.
- **Magnitud del Espacio:** Para la mayoría de problemas NP-completos o combinatorios, el tamaño del espacio de búsqueda sin restricciones crece a ritmo exponencial ($O(b^d)$) o factorial ($O(n!)$), donde b es el factor de ramificación y d es la profundidad máxima de decisión.

Construcción de Soluciones

La construcción de soluciones en Backtracking sigue una estructura metódica dividida en etapas. Formalmente, una solución se expresa como un vector de decisiones:

$$S = (x_1, x_2, \dots, x_n)$$

Donde cada componente x_i es una decisión tomada en la etapa i perteneciente a un conjunto de opciones válidas D_i .

Proceso Paso a Paso en la Etapa k :

- 1. Selección:** Se escoge un elemento candidato x_k en D_k para la posición actual.
- 2. Validación (Función Prometedora):** Se verifica si la solución parcial $(x_1, x_2, \dots, x_k)$ cumple con las restricciones del problema.
- 3. Avance (Recursión):**
Si el estado es válido y $k < n$, el algoritmo avanza recursivamente a la etapa $k+1$.
Si $k = n$, se ha completado la construcción y se procesa/almacena la solución hallada.
- 4. Deshacer (Vuelta Atrás):** Al retornar de la llamada recursiva, se retira x_k del vector de estado, restaurando las variables a su condición previa para evaluar el siguiente elemento del dominio D_k .

Mecanismo de Poda (Pruning)

La poda es el mecanismo decisivo de optimización en el algoritmo de Backtracking. Consiste en suspender la exploración a lo largo de una rama del árbol de decisiones en cuanto se determina que dicha subrama no puede conducir a una solución válida u óptima.

- **Función de Restricción / Poda:** Se aplica mediante predicados lógicos evaluados en cada nodo intermedio. Si una solución parcial viola al menos una regla fundamental del problema, la rama descendiente se considera "no prometedora".
- **Impacto en el Árbol de Búsqueda:** La poda elimina subárboles completos antes de recorrerlos. Si una violación se detecta en la profundidad k , se evita la generación y análisis de hasta $b^{(n-k)}$ estados descendientes.
- **Tipos Principales de Poda:**
 - Poda por Inviabilidad:** Abandona la búsqueda si la solución parcial actual rompe una regla estricta (ej. colocar dos fichas en la misma posición prohibida).
 - Poda por Optimidad (Branch & Bound):** En problemas de optimización, abandona una rama si el costo acumulado de la solución parcial supera la mejor cota obtenida hasta ese momento.

Fuerza Bruta vs. Backtracking

Aunque ambas estrategias exploran el espacio de soluciones para encontrar resultados exactos, difieren fundamentalmente en el momento y la forma en que evalúan las restricciones.

VS.	<i>Fuerza Bruta</i>	<i>Backtracking</i>
Estrategia de Generación	Genera la combinación completa $S = (x_1, \dots, x_n)$ antes de verificar si es válida.	Construye la solución elemento por elemento x_k y verifica la validez en cada etapa intermedia.
Evaluación de Restricciones	Tardía: Comprueba las reglas únicamente en los nodos hoja (soluciones completas).	Temprana: Comprueba las reglas en nodos intermedios (soluciones parciales).
Manejo de Ramas Inválidas	Explora todos los descendientes de una rama, aunque sus primeros elementos rompan las reglas.	Aplica poda inmediata al detectar un estado no prometedor, descartando la subrama completa.
Nodos Explorados	Evalúa la totalidad del árbol de posibilidades (N_{bruto}).	Evalúa un subconjunto drásticamente menor del árbol ($N_{\text{podado}} \ll N_{\text{bruto}}$).
Complejidad Práctica	Siempre alcanza el peor caso teórico ($O(b^n)$ o $O(n!)$).	Aunque su peor caso teórico sea el mismo, el tiempo de ejecución promedio se reduce considerablemente.

ALGORITMOS

Para mejor explicación de backtracking, se propondrán cuatro problemas con sus algoritmos de solución los cuales son:

- Problema de las permutaciones
- Problema de las combinaciones
- Problema de las N-reinas
- Problema del Sudoku

Se mostrará el programa con el problema resuelto

Programa que genera N permutaciones:

```
=====
ALGORITMO DE PERMUTACIONES
=====

Ingrese el valor de N : 3

Generando permutaciones para N = 3:

Solucion 1: [ 1 2 3 ]
Solucion 2: [ 1 3 2 ]
Solucion 3: [ 2 1 3 ]
Solucion 4: [ 2 3 1 ]
Solucion 5: [ 3 1 2 ]
Solucion 6: [ 3 2 1 ]

Total de permutaciones generadas para N = 3: 6

-----
Process exited after 10.24 seconds with return value 0
Presione una tecla para continuar . . . █
```

Programa para generar y explorar combinaciones de N elementos tomados en grupos de K:

```
=====
ALGORITMO PARA OBTENER COMBINACIONES
=====

Ingrese el valor de N (total de elementos, 1-15): 4
Ingrese el valor de K (tamaño del subconjunto, 1-4): 3

Generando combinaciones de 4 elementos tomados de 3 en 3:

Combinacion 1: [ 1 2 3 ]
Combinacion 2: [ 1 2 4 ]
Combinacion 3: [ 1 3 4 ]
Combinacion 4: [ 2 3 4 ]

Total de combinaciones generadas: 4

-----
Process exited after 13.36 seconds with return value 0
Presione una tecla para continuar . . .
```

Programa que determina la posición de N reinas en un tablero de ajedrez sin que se amenacen entre sí.

```
=====
PROBLEMA DE N-REINAS EN TABLERO DE AJEDREZ (8x8)
=====

Ingresa el numero de reinas a colocar (1 a 8): 8

Buscando formas de colocar 8 reina(s) en el tablero de ajedrez 8x8...

--- Solucion 1 ---
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . . .

--- Solucion 2 ---
Q . . . . . . .
. . . . . Q . .
. . . . . . . Q
. . Q . . . . .
. . . . . Q .
. . . Q . . . .
. Q . . . . .
. . . . . Q . .

--- Solucion 3 ---
```

Programa que resuelve un sudoku:

```

  ALGORITMO PARA RESOLUCION DE UN SUDOKU
=====

SUDOKU INICIAL:

5 3 . | . 7 . | . . .
6 . . | 1 9 5 | . . .
. 9 8 | . . . | . 6 .
-----
8 . . | . 6 . | . . 3
4 . . | 8 . 3 | . . 1
7 . . | . 2 . | . . 6
-----
. 6 . | . . . | 2 8 .
. . . | 4 1 9 | . . 5
. . . | . 8 . | . 7 9

SUDOKU RESUELTO CON EXITO:

5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
-----
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
-----
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9

```

Complejidad computacional de los algoritmos:

Permutaciones de N elementos

COMPLEJIDAD TEMPORAL: $O(N! \cdot N)$

Demostración:

- El árbol de decisiones genera N ramas en el primer nivel, $N-1$ en el segundo, y así sucesivamente hasta llegar a las hojas. El número total de soluciones o permutaciones generadas es $N!$.
- En cada hoja completa (caso base), el algoritmo recorre el arreglo de tamaño N para imprimir o guardar la combinación, lo cual toma tiempo $O(N)$.
- **Propiedad de multiplicación de Big-O:** Al multiplicar la cantidad de soluciones por el costo de procesar cada una: Tiempo Total = $O(N!) \cdot O(N) = O(N! \cdot N)$

COMPLEJIDAD ESPACIAL: $O(N)$

Demostración:

- **Pila de llamadas (Call Stack):** La profundidad máxima de la recursión corresponde a la altura del árbol de decisión, que es N
- **Memoria auxiliar:** Los arreglos de estado (`resultado[]` y `usado[]`) ocupan espacio proporcional a N
- **Propiedad de constantes de Big-O:** $O(N) + O(N) = O(2N) = O(N)$.

Combinaciones (N elementos tomados de K en K)

COMPLEJIDAD TEMPORAL:

$$O\left(\binom{N}{K} \cdot K\right)$$

Demostración:

- El número de hojas o respuestas válidas construidas en el árbol es el coeficiente binomial
- Para cada combinación generada de tamaño K , procesar o imprimir la respuesta toma un tiempo de $O(K)$.
- **Propiedad de multiplicación de Big-O:** Tiempo Total = $O\left(\binom{N}{K}\right) \cdot O(K) = O\left(\binom{N}{K} \cdot K\right)$

COMPLEJIDAD ESPACIAL: $O(K)$

Demostración:

- La profundidad máxima de la llamada recursiva en la pila de memoria está acotada por K (el tamaño de la combinación deseada).
- El arreglo auxiliar para almacenar la combinación utiliza espacio proporcional a K .

N -Reinas (Tablero de 8×8)

COMPLEJIDAD TEMPORAL: $O(N!)$

Demostración:

En la primera fila se prueban N posiciones. En la segunda fila, la función de poda descarta columnas y diagonales, dejando a lo sumo $N-2$ opciones válidas; en la tercera fila a lo sumo $N-4$, y así sucesivamente.

Aunque la **poda** elimina ramas tempranas en el árbol de decisión, la cota superior en el peor de los casos sigue estando acotada factorialmente por $N!$: Tiempo Total = $O(N!)$

Nota : Para un tablero general de $N \times N$, la complejidad es $O(N!)$ en tiempo y $O(N)$ en memoria(espacio). Sin embargo, para el tablero clásico de ajedrez, las dimensiones son constantes (8×8), por lo que el tiempo de ejecución y espacio en pila quedan acotados a $O(1)$

COMPLEJIDAD ESPACIAL: $O(N)$

Demostración:

- Se utiliza un arreglo unidimensional `tablero[N]` de tamaño N (donde el índice es la fila y el valor guardado es la columna) en lugar de una matriz de $N \times N$, reduciendo el consumo de memoria.
- La pila recursiva alcanza una profundidad máxima de N llamadas (una por cada fila).
- **Propiedad de Big-O:** Sumando el arreglo y la pila: $O(N) + O(N) = O(N)$.

Sudoku (9×9)

COMPLEJIDAD TEMPORAL: $O(9^M)$ (DONDE M ES EL NÚMERO DE CASILLAS VACÍAS, $M \leq 81$)

Demostración:

- En el peor escenario hipotético (fuerza bruta sin validación), para cada una de las M casillas vacías se probarían 9 dígitos (9^M)
- La **poda** (evaluación de reglas de fila, columna y subcuadrícula 3×3) elimina de inmediato la mayoría de los caminos inviables, ejecutándose en milisegundos. Sin embargo, la cota teórica máxima se expresa como: Tiempo Peor Caso = $O(9^M)$

COMPLEJIDAD ESPACIAL: $O(M)$ O $O(1)$

Demostración:

- La memoria de la matriz 9×9 es fija.
- La profundidad de la pila de recursión es a lo sumo M (la cantidad de celdas vacías a llenar, máximo 81).
- Al ser el tamaño del tablero una constante fija (81 celdas), el espacio auxiliar se considera constante $O(1)$.

APLICACIONES DEL ALGORITMO DE BACKTRACKING

Usos de los algoritmos:

Permutaciones:

- **Ruteo y Logística:** Base para resolver el Problema del Vendedor Viajero en grafos pequeños, evaluando todos los ordenamientos posibles de ciudades para encontrar la ruta óptima.
- **Criptanálisis:** Pruebas de fuerza bruta y ataques por diccionario sobre combinaciones de claves secuenciales.

Combinaciones:

- **Optimización de Recursos (Mochila):** Selección de subconjuntos de elementos que maximicen el valor sin exceder la capacidad de peso.
- **Pruebas de Software:** Generación de subconjuntos de parámetros de entrada para probar interacciones entre componentes sin ejecutar todas las permutaciones posibles.

N-Reinas:

- **Diseño de Circuitos Integrados:** Posicionamiento de componentes o buses de datos en chips evitando interferencias cruzadas o sobrecalentamiento en filas, columnas o diagonales.
- **Planificación de Tareas Multiprocesador:** Asignación de procesos a núcleos de procesamiento garantizando que no existan conflictos de recursos compartidos simultáneos.

Sudoku:

- **Motores SAT Solvers:** La lógica de validación de filas, columnas y bloques sirve como modelo didáctico para los motores de Satisfacibilidad Proposicional (SAT), empleados en la verificación formal de microprocesadores.
- **Asignación de Frecuencias en Telecomunicaciones:** Asignación de canales de radiofrecuencia a celdas contiguas garantizando la ausencia de interferencia en subredes.

Comparación entre algoritmos:

ALGORITMO	PERMUTACIONES	COMBINACIONES
Complejidad temporal	$O(N! \cdot N)$	$O\left(\binom{N}{K} \cdot K\right)$
Complejidad espacial	$O(N)$	$O(K)$
Dificultad de implementación	Sencilla	Sencilla
Ventajas	Garantiza la exploración completa de secuencias ordenadas.	Genera únicamente subconjuntos únicos reduciendo redundancia.
Desventajas	Explosión combinatoria rápida para $N > 10$	Requiere gestionar múltiples índices de control (N y K).
Situaciones recomendadas	Secuenciación de procesos u ordenamiento de rutas.	Selección de grupos o filtrado de subconjuntos de datos.

ALGORITMO	N-REINAS	SUDOKU
Complejidad temporal	$O(N!)$ general vs $O(1)$ para 8x8	$O(9^M)$, peor caso ($M \leq 81$)
Complejidad espacial	$O(N)$ general vs $O(1)$ en 8x8	$O(M)$ vs $O(1)$ si es constante
Dificultad de implementación	Media	Media-sencilla
Ventajas	Excelente ejemplo de representación reducida de memoria	Altamente eficiente; la poda elimina más del 99% del espacio inútil de búsqueda.
Desventajas	La validación diagonal requiere cálculos de distancia absoluta (abs).	Matriz rígida; si el tablero no tiene solución previa, explora el peor caso completo.
Situaciones recomendadas	Asignación de recursos con restricciones espaciales/diagonales.	Problemas de cuadrícula con restricciones múltiples por zona/bloque.

Otras aplicaciones:

Problemas de Satisfacción de Restricciones (CSP)

- **Definición y Alcance:** Los problemas CSP buscan asignar valores a un conjunto de variables sujeto a reglas o restricciones específicas. Backtracking es la técnica por excelencia para explorar el espacio de estados descartando asignaciones parciales inconsistentes de manera temprana.
- **Ejemplos Típicos:** Motores de inferencia en Inteligencia Artificial, acertijos lógicos (Sudoku, Kakuro) y problemas de asignación eficiente de recursos.

Coloreado de Grafos (Graph Coloring)

- **Asignación de Frecuencias en Telecomunicaciones:** En redes inalámbricas y móviles, se modela cada antena como un vértice y las interferencias entre coberturas adyacentes como aristas. El Backtracking asigna la menor cantidad posible de frecuencias (colores) garantizando que antenas contiguas no interfieran entre sí.
- **Planificación de Horarios y Exámenes:** Mapea asignaturas como vértices y conflictos de horario/estudiantes como aristas, asegurando que no existan solapamientos de aulas ni cruces para alumnos o profesores.

Búsqueda de Rutas y Navegación (Pathfinding y Laberintos)

- **Robótica y Navegación Autónomas:** Permite a un agente móvil o robot encontrar un camino válido desde un punto de origen hasta una meta en entornos cerrados o tableros de navegación, retrocediendo ante obstáculos o caminos sin salida.
- **Recorrido de Grafos y Juegos (Knight's Tour / Caminos Hamiltonianos):** Utilizado en teoría de grafos para hallar ciclos hamiltonianos y en IA de videojuegos para validar conectividad entre áreas o rompecabezas de mapas.

Criptoaritmética y Criptografía

- **Acertijos Criptoaritméticos:** Resolución de ecuaciones de letras por dígitos (como SEND + MORE = MONEY) probando permutaciones válidas de dígitos mediante exploración guiada y poda de inconsistencias matemáticas.
- **Análisis y Descifrado:** Evaluación de estructuras de claves y combinaciones en esquemas de seguridad combinatoria.

Pruebas Automáticas y Verificación de Software

- **Generación de Casos de Prueba (Automated Test Generation):** Exploración de las distintas ramas de ejecución de un programa informático para encontrar entradas que maximicen la cobertura de código o detecten estados de falla (bugs).
- **Verificación Formal de Sistemas:** Comprobación de que una máquina de estados o protocolo cumple con especificaciones de seguridad sin caer en bloqueos mutuos (deadlocks).

BIBLIOGRAFIA

- Brassard, G., & Bratley, P. (1997). Fundamentos de Algoritmos. Prentice Hall.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- Horowitz, E., Sahni, S., & Rajasekaran, S. (2008). Fundamentals of Computer Algorithms (2nd ed.). Silicon Press.
- Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.